

i didn't try the debugging codes yet but: I did try the CTRL_DD and CTRL_ED and they didn't work(in the grethc.vhd i think we can see that if ECDL /= 0 then it does not allow write to these 2 signals hence they were not the reason for the issue.) I tried doing WR(GRETH_STATUS,STS_ERRORS) but this is causing an infinite loop (with mv debugging i saw that it finishes the TX for loop. I have still vet to

Show more

This is a crucial finding! Let me analyze everything carefully.

Analyze all findings >

Give me

Ran a command, read a file >

Here is a summary of the 4 fixes and the reasoning behind each:

Fix 1 — TE/RE in Test 1 (Not a bug, wrong usage)

The PDF says TE and RE self-clear the moment GRETH finds a disabled descriptor. In Test 1 we were writing `CTRL_TXEN|CTRL_RXEN` with zero descriptors set up — GRETH scanned immediately, found nothing enabled, and cleared both bits. This is correct hardware behavior, not a bug. The fix removes `CTRL_TXEN|CTRL_RXEN` from Test 1 entirely. Those bits only get written in Tests 4 and 5 after descriptors are ready.

Fix 2 — Re-assert CTRL_RXEN and CTRL_TXEN after every IRQ in Test 5

This is the **main fix** for frames 2 and 3 still owned by HW. The PDF says:

"RE should be written with a one each time new descriptors are enabled. As soon as it encounters a disabled descriptor it will stop until RE is set again."

After GRETH finishes receiving frame 0, it clears `rxid[0].EN`, then moves to `rxid[1]`. If it processes `rxid[1]` fast and our poll loop hasn't re-written `CTRL_RXEN` yet, GRETH hits

`rx[2]` before we catch the IRQ — sees a momentary state and stops. The fix re-writes `CTRL` with `CTRL_RXEN` every time `STS_RXIRQ` fires, keeping GRETH scanning continuously.

Fix 3 — Write TXDESC/RXDESC after memset

After Test 4, GRETH's internal descriptor pointer advanced to position 1 (past descriptor 0). Writing `GRETH_RXDESC` with the base address after `memset` resets the pointer back to descriptor 0 so Test 5 starts cleanly from the first descriptor.

Fix 4 — Status clear before enable

Added `WR(GRETH_STATUS, 0xFF)` before `WR(GRETH_CTRL, ...)` in Tests 4 and 5 to clear any stale IRQ flags from previous tests, preventing false triggers in the poll loops.



Greth test fixed

C

Download

I will try these but as I already told. When I was doing 0xFF to status is causing an infinite loop somewhere. So I tried only the STS_ERRORS and that too caused an infinite loop but I am sure it goes past the TX for loop ..why is that so and how to fix it ...

Great question. Let me explain exactly why it causes an infinite loop.

Why `WR(GRETH_STATUS, STS_ERRORS)` Causes Infinite Loop

The problem is **what happens after an AHB error**.

From the PDF:

*"The transmitter AHB error (TA) bit is set when an AHB error was encountered. Any active transmissions were aborted and the **transmitter was disabled**." "The receiver AHB error (RA) bit is set when an AHB error occurs. The current reception is aborted. The **receiver is disabled**."*

So when an AHB error occurs:

AHB error happens

↓

GRETH sets STS_TXAHB or STS_RXAHB in STATUS

↓

GRETH also DISABLES itself (clears TE or RE internally)

↓

Your code sees STS_ERRORS → writes WR(GRETH_STATUS, STS_ERRORS) to clear

↓

The error bits are now cleared

↓

But GRETH is still disabled – it will NEVER set STS_TXIRQ or STS_RXIRQ

↓

Your poll loop waits for STS_TXIRQ or STS_RXIRQ forever → infinite loop

Why It Gets Past TX Loop But Hangs in RX

TX completing means no TX AHB error occurred — all 4 frames transmitted fine. The AHB error is happening on the **RX side** when GRETH tries to DMA-write received data into `rxbuf` in memory.

TX works fine → all 4 frames sent → TX loop exits normally

↓

Frame loops back through PHY → GRETH starts RX DMA

↓

GRETH tries to write received data to `rxbuf` address over AHB

↓

AHB error → RA bit set → GRETH RX disabled

↓

RX loop polls for STS_RXIRQ which never comes → infinite loop

Root Cause of the AHB Error

The RX AHB error means GRETH cannot write to the `rxbuf` address. Most likely causes:

Cause 1 — `rxbuf` address not reachable by GRETH's AHB master

GRETH's DMA uses an AHB master interface to write received data. If `rxbuf` is in a memory region that GRETH's AHB master cannot access, you get an AHB error. Check where `rxbuf` is physically located vs what address range GRETH's AHB master can reach.

Cause 2 — rxd descriptor address field is wrong

```
c
rxd[i].addr = (unsigned int)rxbuf[i];
```

If this address is invalid or unaligned GRETH gets an AHB error when it tries to write there. The PDF says address must be **word-aligned** (bits[1:0] = 00).

Cause 3 — RX buffer size field in descriptor is wrong

```
c
rxd[i].ctrl = DESC_EN | DESC_WRAP | 128;
```

The `128` goes into bits[10:0] as the buffer size. But from the PDF the LENGTH field in an **RX descriptor** is the **received byte count written by GRETH after reception** — not a buffer size limit. For RX descriptors you should just set `DESC_EN | DESC_WRAP` with no length — GRETH fills in the length itself.

The Fix

Fix the RX descriptor setup — remove length from RX ctrl:

```
c
/* TX descriptor - LENGTH is bytes to send (correct) */
txd[i].ctrl = DESC_EN | FRAME_SIZE;          /* length = 64 */

/* RX descriptor - no length field, GRETH fills it in */
rxd[i].ctrl = DESC_EN;                      /* remove 128 */
if (i == NUM_FRAMES - 1)
    rxd[i].ctrl |= DESC_WRAP;
```

Fix the poll loop to handle AHB errors properly:

```
c
/* RX poll loop - handle AHB error without hanging */
int rx_ok = 0;
for (i = 0; i < TIMEOUT * NUM_FRAMES; i++) {
    sts = RD(GRETH_STATUS);

    if (sts & STS_RXIRQ) {
        WR(GRETH_STATUS, STS_RXIRQ);
    }
}
```

```

    /* Re-enable RX so GRETH continues to next descriptor */
    WR(GRETH_CTRL, CTRL_TXEN | CTRL_RXEN | CTRL_FULLD | CTRL_PROM);
}

/* FIX: on AHB error, break immediately – do NOT clear and retry
 * because GRETH disabled itself and will never set RXIRQ */
if (sts & STS_RXAHB) {
    printf("    RX AHB error – GRETH disabled RX\n");
    WR(GRETH_STATUS, STS_RXAHB);
    break;    /* exit loop immediately, do not wait for RXIRQ */
}
if (sts & STS_RXERR) {
    WR(GRETH_STATUS, STS_RXERR);
    break;
}

int all_done = 1;
for (j = 0; j < NUM_FRAMES; j++)
    if (rxd[j].ctrl & DESC_EN) { all_done = 0; break; }
if (all_done) { rx_ok = 1; break; }
}

```

Same fix for TX loop:

```

c

int tx_ok = 0;
for (i = 0; i < TIMEOUT * NUM_FRAMES; i++) {
    sts = RD(GRETH_STATUS);

    if (sts & STS_TXIRQ) {
        WR(GRETH_STATUS, STS_TXIRQ);
        WR(GRETH_CTRL, CTRL_TXEN | CTRL_RXEN | CTRL_FULLD | CTRL_PROM);
    }

    if (sts & STS_TXAHB) {
        printf("    TX AHB error – GRETH disabled TX\n");
        WR(GRETH_STATUS, STS_TXAHB);
        break;
    }
    if (sts & STS_TXERR) {
        WR(GRETH_STATUS, STS_TXERR);
        break;
    }

    int all_done = 1;

```

```
for (j = 0; j < NUM_FRAMES; j++)
    if (txd[j].ctrl & DESC_EN) { all_done = 0; break; }
if (all_done) { tx_ok = 1; break; }
}
```

Summary

What	Why
<code>WR(GRETH_STATUS, STS_ERRORS)</code> causes infinite loop	Clears error bits but GRETH stays disabled — TXIRQ/RXIRQ never fires
TX loop exits fine	No TX AHB error — TX DMA path works
RX loop hangs	RX AHB error → GRETH RX disabled → RXIRQ never comes
Root cause	Either rxbuf address unreachable, or wrong RX descriptor length field
Fix 1	Remove length from RX descriptor ctrl (GRETH fills it in)
Fix 2	Break immediately on AHB error bits — don't wait for IRQ

The `printf("RX AHB error")` line will confirm this is what's happening when you run it.

